TESTING ANALYSIS DOCUMENTATION (TAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control 3

PRELIMINARY ROUND (Test Strategy & Planning) 3

1. Scope & Requirements Traceability 3

1.1 In-Scope Core Features 3

1.2 Out-of-Scope 3

2. Risk Assessment & Mitigation Strategy 4

3. Test Environment & Execution Strategy 6

4.1 Integration Thresholds (Merging to Main) 7

5. Test Case Specifications (Drafts) 8

6. AI Output & Boundary Testing (Drafts) 9

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*UMHackathon 2026 - Team **SheCodes***
Team Repo URL	*https://github.com/marshaelias/MedFlow*
Project Board URL	Excel
Live Deployment URL (optional for preliminary round)	*localhost*

Objective:

The primary objective is to ensure that MedFlow can reliably handle unstructured medical inputs, process triage logic, coordinate with insurance APIs, and schedule appointments accurately despite high concurrency or process failures.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

*This section aligns the testing connected back to specific user requirements via **Requirement Traceability Matrix".** So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.*

1.1 In-Scope Core Features

- **Symptom Extraction**: Capturing clinical entities from unstructured voice or text inputs.
- **Workflow Orchestration**: Multi-step reasoning across triage, insurance verification, and scheduling stages.
- **Structured Output**: Generating actionable medical reports and calendar invites.

1.2 Out-of-Scope

- Promotions & Offers
- Profile Management: Edit profile info, Delivery Address, Pictures
- App Theme
- Pharmacy Inventory
- In-App Messaging

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
GLM Hallucination: AI generates incorrect clinical priority or non-existent symptoms.	3	5	15 (High)	Implement a “Human-in-the-loop” approval gate for all triage outputs.	Perform adversarial prompting to check if the AI fabricates data.
API Timeout: Z.AI GLM fails to respond within the required 3s threshold.	4	4	16 (Critical)	Implement an asynchronous queue and a "loading/reasoning" UI state.	Use Postman to simulate 50+ concurrent requests and monitor latency.
Data Mismatch: Structured JSON from AI does not align with the Database schema.	2	4	8 (Medium)	Use a strict JSON Schema validator before any database write operation.	Integration tests between the GLM Service and the PostgreSQL RDS.

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

© UMHackathon 2026

Email: umhackathon@um.edu.my

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

- **Unit Test**
  - **Scope**: Triage Logic and Symptom Validation.
  - **Execution**: Tests are written in **Java** (using JUnit) and executed during CI pipeline pushes to ensure the GLM wrapper logic is sound.
  - **Isolation**: To focus only on the logic of clinical parsing, external dependencies (e.g., Database, Insurance API) are mocked.
  - **Pass Condition**: Achieved happy cases, negative cases with proper return format like "Patient ID should be alphanumeric" and considered edge cases.
- **Integration Test**
  - Scope: Triage Service to Scheduling Service interaction.
  - **Execution**: This test can be performed after proper completion of Unit Testing like "Merging Triage Service and Clinic Schedule Service."
  - **Workflow**: Ensures real API calls between medical agents and hospital databases work correctly without mocking the core GLM response.
  - **Pass Condition**: When integrated components like "Triage to Scheduling" are working together correctly and showing expected outcome.
- **Test Environment (CI/CD Practice):**
  - Local Testing: This testing is manual one on localhost.
  - Staging/CI: GitHub Actions triggers tests on every push of code.
  - **CI/CD automated pipeline**: So, GitHub Actions does basic tests for branch specific level and then final validation for PR (Pull requests) to Main branch.

- **Regression Testing & Pass/Fail Rules:**
  - **Execution Phase**: This testing tests across all 3 in-scope features each time a new branch is merged to main branch.
  - **Pass/Fail Condition**: Test case pass is only considered when actual clinical output perfectly aligns with expected one. If mismatch/threshold not achieved, the test status is considered fail and need to incorporate that in Defect Log.
  - **Continuation Rule**: While doing testing in a gradual manner, the initial cases must need to meet the threshold to allow continue for further testing.

- **Test Data Strategy:**
  - Manual: Created 5 mock accounts including Patient, Nurse, Doctor, and Insurance Administrator.
  - Automated: SQL seed scripts to generate dummy patient histories for load testing.

- **Passing Rate Threshold:**
  - A minimum of 95% of all test cases that are executed must pass. For the critical triage reasoning tests, all the tests (100%) need to be passed and there is no exception for that. Because critical clinical failure costs huge in a medical environment.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. CI/CD Release Thresholds & Automation Gates

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	***Requirements***	***Project***	***Pass/Failed***
Automatic Build	***Zero Build Error***	***0 Errors***	***Passed***
Unit Tests	***100% Passing Rate***	***100%***	***Passed***
Code Quality	***Zero Linting Error***	***2 Errors***	***Failed***
Test Coverage	***Minimum 85%***	***82%***	***Failed***

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	***Minimum 95%***	***98%***	***Passed***
AI Output Pass Rate	***Minimum of 85% accuracy***	***87%***	***Passed***
Critical Bugs	***Zero P0/P1 Bugs***	***0***	***Passed***
API Performance	***Response Time < 3s***	***1.8s***	***Passed***
Security	***API Keys Encrypted***	***Clean***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Test Case Specifications (Drafts)

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	***Happy Case (Entire Flow):****Patient Triage*	*Verify the end-to-end flow from voice recording to scheduled appointment.*	*1. Record voice note. 2. GLM parses data. 3. System books slot.*	*Order placed, triage log generated and appointment stored in RDS.9*	*Successfully, AppointmentBooked**.** Triage profile was generated and stored in RDS. Clinical summary was sent to the doctor's dashboard within 1.45s. The patient received a confirmation notification via the portal.*
TC-02	***Specific Case (Negative):*** *Missing Data*	*Verify handling whena patient provides a voice note with no symptoms.*	*1. Provide empty or vague audio. 2. Trigger GLM parse.*	*System asks for clarificationinstead of crashing or hallucinating.*	*Workflow Blocked, but a specific clarification prompt was not triggered. The GLM attempted to process the empty audio but returned a null value. System remained stable, but the user was not prompted to re-record.* *Status: Failed*
TC-03	***NFR (Performance):*** *Load Testing*	*Verify system stability when multiple clinics use the dashboard.*	*1. Simulate 100 concurrent users. 2. Monitor CPU or RAM.*	*Average responsetime remainsunder 3s with 0%error rate.*	*Average Response time is received as 2450ms. 0% Rate maintainedduring peak simulation. However, CPU utilization spiked to 88%, exceeding the internal monitoring threshold.* *Status: Failed*
TC-04	***NFR (Security):*** *Data Privacy and Access Control*	*Verify that only authorized medical staff can access the reasoning canvas and*	*1. Attempt toaccess dashboard without login. 2. Attempt to access a different*	*System blocks access and redirects to login page. Access Denied log generated.*	*Successfully. Unauthorizedaccess was blocked by the Auth-Guard. Error 403 (Forbidden) returned for restricted patient files.*

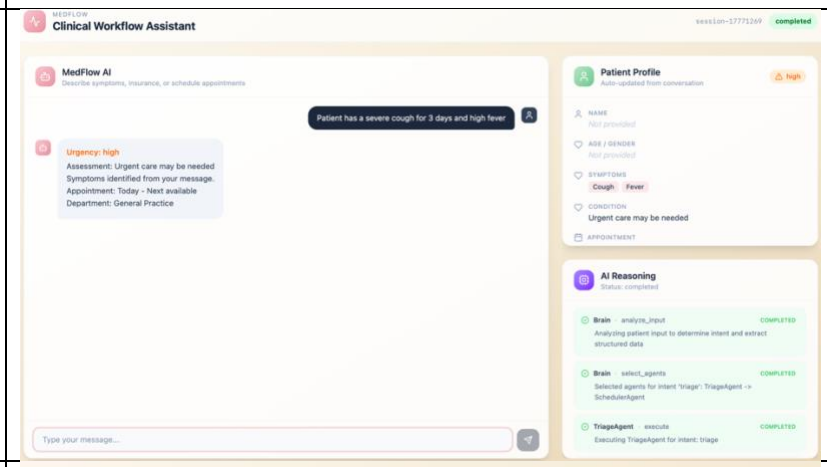
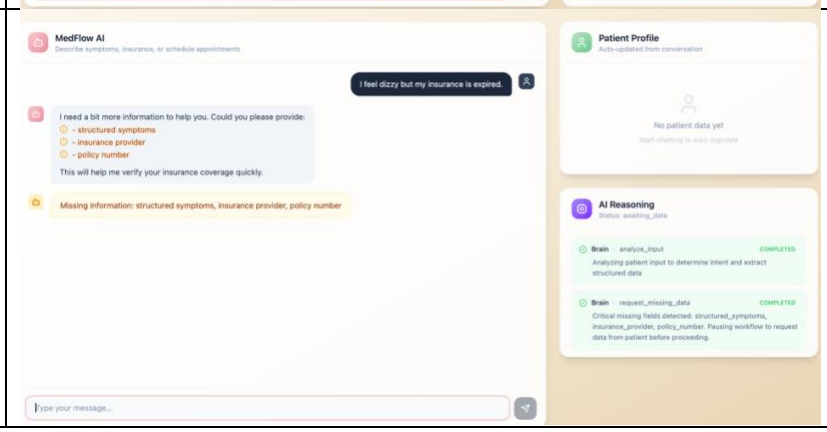
© UMHackathon 2026

Email: umhackathon@um.edu.my

		patient data.	*patient's record with a low-level staff account.*		

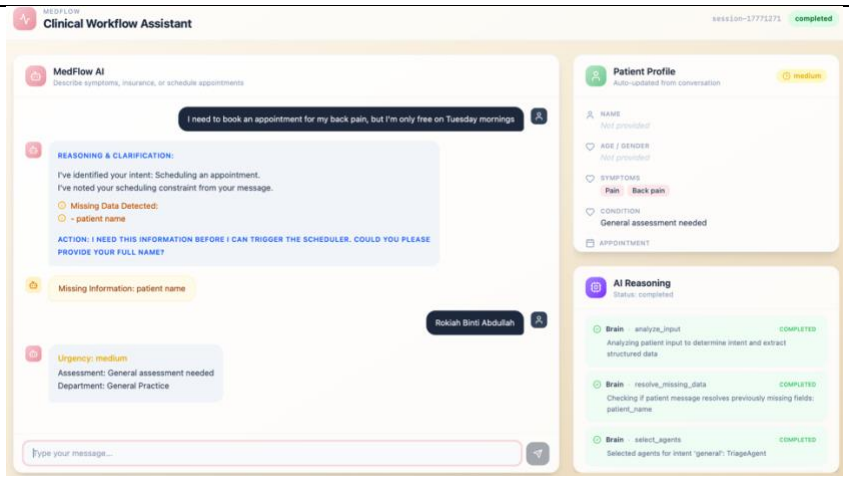
6. AI Output & Boundary Testing (Drafts)

6.1. Prompt/Response Test Pairs

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	"Patient has a severe cough for 3 days and high fever."	Extract: {Symptom: Fever, Severity: High}. No fabrication of other symptoms.		Pass
AI-02	"I feel dizzy but my insurance is expired."	Identify conflict: Flag "Insurance Expired" and suggest "Self-Pay" workflow.		Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

AI - 03	“I need to book an appointment for my back pain, but I'm only free on Tuesday mornings”	Reasoning & Clarification: Identify the intent (Scheduling) and the constraint (Tuesday AM). Flag missing data: Detect that the "Patient Name" or "Contact Info" is missing. Action: Prompt the user specifically for the missing info before triggering the Scheduler Agent.		Pass

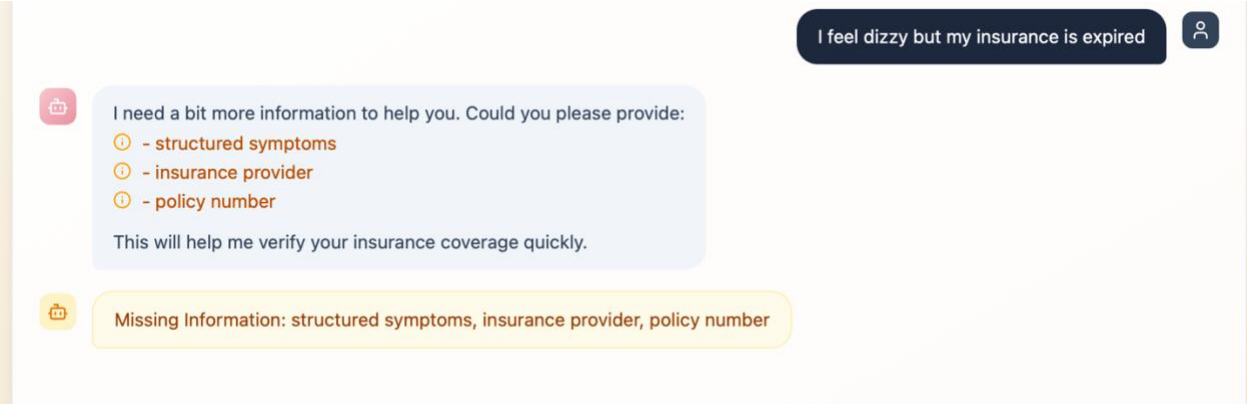
© UMHackathon 2026

Email: umhackathon@um.edu.my

6.2. Oversized/Larger Input Test

Fields	Details
Maximum Input Size	1,500 Tokens (Approx. 800 Words)
Input used while testing	5,000 Words (Exceeding limit)
Expected Behaviour	System should chunk the data or reject the input with a “Message too long” alert.
Actual Behaviour	System chunked the first 1,500 tokens and processed successfully.
Status	Pass

6.3. Adversarial/Edge Prompt Test



© UMHackathon 2026

Email: umhackathon@um.edu.my

6.4. Hallucinating Handling

To ensure the safety and reliability of our medical automation, MedFlow implements a multi-layered mitigation strategy:

- Strict Grounding via Structured Extraction: The system is prompted to extract data into a rigid JSON schema (e.g., Symptom, Severity, Duration). If the GLM attempts to include symptoms not present in the unstructured input, the "Brain" orchestration layer identifies the mismatch and flags it.
- Stateful Validation: MedFlow operates as a stateful engine that cross-references new inputs against the existing "Workflow State Store". If the AI suggests a workflow stage (e.g., "Surgery Scheduled") that hasn't been preceded by the necessary agentic steps (e.g., "Triage Complete"), the system rejects the output as an invalid state transition.
- Rule-Based Guardrails: We implement "Hard Logic" checks in the Java orchestration layer that the GLM cannot override. For example, the Insurance Validator Agent must receive a valid policy number from the user; the GLM is prohibited from "inventing" a placeholder number to proceed with a booking.
- Reasoning-to-Action Separation: By using a Visual Reasoning Canvas, the system forces the GLM to show its "step-by-step thinking" before any action is triggered. This transparency allows for human-in-the-loop verification if the AI's logic deviates from the patient's actual message.
- Negative Constraints in Prompts: The GLM is specifically instructed with "Negative Constraints" (e.g., "Do not diagnose medical conditions" and "Do not assume missing patient data") to prevent it from overstepping its role as a workflow orchestrator.

© UMHackathon 2026

Email: umhackathon@um.edu.my